

Prompt B — SPA: Autocomplete Habitación, Clientes SPA y Confirmación PDF

PARTE 1 — Autocomplete datos del huésped al seleccionar habitación

El problema

Hoy cuando se selecciona una habitación en el formulario de nuevo turno, los campos de nombre, apellido, teléfono y email quedan vacíos. Hay que autocompletarlos con los datos del huésped de esa reserva.

Fix en `spa.tsx`

El formulario ya tiene `checkedInReservations` con todos los datos de las reservas (incluyendo `guest`). Solo hay que agregar un `useEffect` o un `onValueChange` en el selector de reserva.

En el `<FormField>` del campo `reservationId`, cambiar el `onValueChange` de:

```
// Actual:
<Select onValueChange={field.onChange} value={field.value || undefined}>
```

A:

```
<Select
  onValueChange={(value) => {
    field.onChange(value);
    if (value) {
      const res = checkedInReservations.find(r => r.id === value);
      if (res?.guest) {
        form.setValue("guestName", res.guest.firstName || "");
        form.setValue("guestLastName", res.guest.lastName || "");
        form.setValue("guestPhone", res.guest.phone || "");
        form.setValue("guestEmail", res.guest.email || "");
      }
    } else {
      // Si se deselecciona, limpiar campos
      form.setValue("guestName", "");
    }
  }}
  value={field.value || undefined}>
```

```

        form.setValue("guestLastName", "");
        form.setValue("guestPhone", "");
        form.setValue("guestEmail", "");
    }
    }}
    value={field.value || undefined}
>

```

Agregar también una nota visual debajo del selector:

```

{form.watch("reservationId") && (
  <p className="text-xs text-muted-foreground mt-1 flex items-center gap-1">
    <CheckCircle2 className="h-3 w-3 text-green-500" />
    Datos del huésped cargados automáticamente
  </p>
)}

```

Agregar `CheckCircle2` a los imports de `lucide-react` si no está.

PARTE 2 — Base de datos de Clientes SPA

2a — Nueva tabla en `shared/schema.ts`

```

export const spaClients = pgTable("spa_clients", {
  id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
  firstName: text("first_name").notNull(),
  lastName: text("last_name"),
  phone: text("phone"),
  email: text("email"),
  notes: text("notes"), // alergias, preferencias, observaciones
  createdAt: timestamp("created_at").notNull().defaultNow(),
});

export const insertSpaClientSchema = createInsertSchema(spaClients).omit({ id: true });
export type InsertSpaClient = z.infer<typeof insertSpaClientSchema>;
export type SpaClient = typeof spaClients.$inferSelect;

```

2b — Migración

```

await db.execute(sql`
  CREATE TABLE IF NOT EXISTS spa_clients (
    id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

    first_name TEXT NOT NULL,
    last_name TEXT,
    phone TEXT,
    email TEXT,
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
)
`);

```

2c — Endpoints

```

app.get("/api/spa/clients", async (req, res) => {
  try {
    const { search } = req.query;
    let query = db.select().from(spaClients);
    const clients = await query.orderBy(desc(spaClients.createdAt));
    if (search) {
      const s = (search as string).toLowerCase();
      return res.json(clients.filter(c =>
        c.firstName.toLowerCase().includes(s) ||
        (c.lastName || "").toLowerCase().includes(s) ||
        (c.phone || "").includes(s) ||
        (c.email || "").toLowerCase().includes(s)
      ));
    }
    res.json(clients);
  } catch { res.status(500).json({ error: "Error" }); }
});

app.post("/api/spa/clients", async (req, res) => {
  try {
    const [created] = await db.insert(spaClients).values(req.body).returning();
    res.json(created);
  } catch { res.status(500).json({ error: "Error" }); }
});

app.patch("/api/spa/clients/:id", async (req, res) => {
  try {
    const [updated] = await db.update(spaClients)
      .set(req.body).where(eq(spaClients.id, req.params.id)).returning();
    res.json(updated);
  } catch { res.status(500).json({ error: "Error" }); }
});

app.delete("/api/spa/clients/:id", async (req, res) => {
  try {

```

```

    await db.delete(spaClients).where(eq(spaClients.id, req.params.id));
    res.json({ success: true });
  } catch { res.status(500).json({ error: "Error" }); }
});

```

2d — Nueva página `client/src/pages/spa-clients.tsx`

```

import { useState } from "react";
import { useQuery, useMutation } from "@tanstack/react-query";
import { Plus, Search, Pencil, Trash2, UserRound, Phone, Mail, FileText } from "lucide-react";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Card, CardContent } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription } from "@components/ui/dialog";
import { Textarea } from "@components/ui/textarea";
import { useToast } from "@hooks/use-toast";
import { apiRequest, queryClient } from "@lib/queryClient";

type SpaClient = {
  id: string;
  firstName: string;
  lastName: string | null;
  phone: string | null;
  email: string | null;
  notes: string | null;
  createdAt: string;
};

export default function SpaClientsPage() {
  const { toast } = useToast();
  const [searchTerm, setSearchTerm] = useState("");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editingClient, setEditingClient] = useState<SpaClient | null>(null);
  const [form, setForm] = useState({ firstName: "", lastName: "", phone: "", email: "" });

  const { data: clients = [], isLoading } = useQuery<SpaClient[]>({
    queryKey: ["/api/spa/clients"],
  });

  const filteredClients = clients.filter(c => {
    const s = searchTerm.toLowerCase();
    return !s || c.firstName.toLowerCase().includes(s) ||
      (c.lastName || "").toLowerCase().includes(s) ||
      (c.phone || "").includes(s) ||
      (c.email || "").includes(s);
  });

  const createClient = async () => {
    const client: SpaClient = {
      firstName: form.firstName,
      lastName: form.lastName,
      phone: form.phone,
      email: form.email,
      createdAt: new Date().toISOString(),
    };
    await apiRequest("POST", "/api/spa/clients", client);
    queryClient.invalidateQueries(["/api/spa/clients"]);
    toast({ title: "Client created", description: "New client added successfully." });
  };

  const updateClient = async (client: SpaClient) => {
    await apiRequest("PUT", `/api/spa/clients/${client.id}`, client);
    queryClient.invalidateQueries(["/api/spa/clients"]);
    toast({ title: "Client updated", description: "Client information updated successfully." });
  };

  const deleteClient = async (id: string) => {
    await apiRequest("DELETE", `/api/spa/clients/${id}`);
    queryClient.invalidateQueries(["/api/spa/clients"]);
    toast({ title: "Client deleted", description: "Client removed from the system." });
  };

```

```

        (c.email || "").toLowerCase().includes(s);
    });

const createMutation = useMutation({
  mutationFn: async (data: typeof form) => {
    const res = await apiRequest("POST", "/api/spa/clients", data);
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/clients"] });
    setDialogOpen(false);
    resetForm();
    toast({ title: "Cliente registrado" });
  },
});

const updateMutation = useMutation({
  mutationFn: async ({ id, data }: { id: string; data: typeof form }) => {
    const res = await apiRequest("PATCH", `/api/spa/clients/${id}`, data);
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/clients"] });
    setDialogOpen(false);
    resetForm();
    toast({ title: "Cliente actualizado" });
  },
});

const deleteMutation = useMutation({
  mutationFn: async (id: string) => {
    await apiRequest("DELETE", `/api/spa/clients/${id}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/spa/clients"] });
    toast({ title: "Cliente eliminado" });
  },
});

const resetForm = () => {
  setForm({ firstName: "", lastName: "", phone: "", email: "", notes: "" });
  setEditingClient(null);
};

const handleEdit = (client: SpaClient) => {
  setEditingClient(client);
  setForm({

```

```

    firstName: client.firstName,
    lastName: client.lastName || "",
    phone: client.phone || "",
    email: client.email || "",
    notes: client.notes || "",
  });
  setDialogOpen(true);
};

const handleSubmit = () => {
  if (!form.firstName.trim()) return;
  if (editingClient) {
    updateMutation.mutate({ id: editingClient.id, data: form });
  } else {
    createMutation.mutate(form);
  }
};

return (
  <div className="p-6 space-y-6" data-testid="page-spa-clients">
    <div className="flex items-center justify-between">
      <div>
        <h1 className="text-3xl font-bold tracking-tight">Clientes SPA</h1>
        <p className="text-muted-foreground">Base de datos de clientes frequent</p>
      </div>
      <Button onClick={() => { resetForm(); setDialogOpen(true); }} data-testid="New Client"
        <Plus className="h-4 w-4 mr-2" /> Nuevo Cliente
      </Button>
    </div>

    <div className="flex items-center gap-4">
      <div className="relative flex-1 max-w-sm">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" type="text"
          <Input
            placeholder="Buscar por nombre, teléfono o email..."
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
            className="pl-10"
            data-testid="input-search-spa-client"
          />
      </div>
      <Badge variant="outline">{clients.length} clientes</Badge>
    </div>

    {isLoading ? (
      <div className="text-center py-12 text-muted-foreground">Cargando...</div>
    ) : filteredClients.length === 0 ? (

```

```

<div className="text-center py-16 text-muted-foreground">
  <UserRound className="h-12 w-12 mx-auto mb-3 opacity-30" />
  <p>{searchTerm ? "No se encontraron clientes" : "No hay clientes regist
</div>
) : (
<div className="grid gap-3 sm:grid-cols-2 lg:grid-cols-3">
  {filteredClients.map(client => (
    <Card key={client.id} data-testid={`card-spa-client-${client.id}`}>
      <CardContent className="p-4">
        <div className="flex items-start justify-between">
          <div className="flex-1 min-w-0">
            <p className="font-semibold truncate">
              {client.firstName} {client.lastName || ""}
            </p>
            {client.phone && (
              <p className="text-sm text-muted-foreground flex items-cent
                <Phone className="h-3 w-3" /> {client.phone}
              </p>
            )}
            {client.email && (
              <p className="text-sm text-muted-foreground flex items-cent
                <Mail className="h-3 w-3" /> {client.email}
              </p>
            )}
            {client.notes && (
              <p className="text-xs text-muted-foreground flex items-star
                <FileText className="h-3 w-3 mt-0.5 shrink-0" /> {client.
              </p>
            )}
          </div>
          <div className="flex gap-1 ml-2">
            <Button variant="ghost" size="icon" className="h-7 w-7" onCli
              <Pencil className="h-3 w-3" />
            </Button>
            <Button variant="ghost" size="icon" className="h-7 w-7" onCli
              if (window.confirm(`¿Eliminar a ${client.firstName} ${clien
                deleteMutation.mutate(client.id);
              }
            </Button>
          </div>
        </div>
      </CardContent>
    </Card>
  )})
</div>

```

```

    })

    <Dialog open={dialogOpen} onOpenChange={ (open) => { if (!open) { setDialogO
      <DialogContent className="max-w-md">
        <DialogHeader>
          <DialogTitle>{editingClient ? "Editar Cliente" : "Nuevo Cliente SPA"}
          <DialogDescription>Registrar cliente frecuente del SPA</DialogDescrip
        </DialogHeader>
        <div className="grid gap-4 py-4">
          <div className="grid grid-cols-2 gap-3">
            <div>
              <Label>Nombre *</Label>
              <Input value={form.firstName} onChange={e => setForm({ ...form, f
            </div>
            <div>
              <Label>Apellido</Label>
              <Input value={form.lastName} onChange={e => setForm({ ...form, la
            </div>
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div>
              <Label>Teléfono</Label>
              <Input value={form.phone} onChange={e => setForm({ ...form, phone
            </div>
            <div>
              <Label>Email</Label>
              <Input type="email" value={form.email} onChange={e => setForm({ .
            </div>
          </div>
          <div>
            <Label>Observaciones (alergias, preferencias...)</Label>
            <Textarea value={form.notes} onChange={e => setForm({ ...form, note
          </div>
        </div>
        <DialogFooter>
          <Button variant="outline" onClick={() => { setDialogOpen(false); rese
          <Button onClick={handleSubmit} disabled={!form.firstName.trim() || cr
            {(createMutation.isPending || updateMutation.isPending) ? "Guardand
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  </div>
  );
}

```


2e — Agregar ruta y menú

En `App.tsx` :

```
import SpaClientsPage from "@pages/spa-clients";
// En el router:
<Route path="/spa-clients" component={SpaClientsPage} />
```

En `AppSidebar` , dentro del grupo donde está el ítem de SPA, agregar debajo:

```
<SidebarMenuItem>
  <SidebarMenuButton asChild isActive={location === "/spa-clients"} data-testid="
    <Link href="/spa-clients">
      <UserRound className="h-5 w-5" />
      <span>Clientes SPA</span>
    </Link>
  </SidebarMenuButton>
</SidebarMenuItem>
```

Agregar `UserRound` a los imports de `lucide-react` del sidebar.

2f — Selector de cliente SPA en el formulario de nuevo turno

Agregar un selector de cliente frecuente al formulario, ANTES de los campos manuales de nombre/apellido. Si se selecciona un cliente, los campos se autocompletan (igual que con habitación):

```
{/* Buscador de cliente SPA frecuente */}
<div>
  <Label className="text-sm font-medium">Cliente frecuente (opcional)</Label>
  <SpaClientSelector
    onSelect={(client) => {
      form.setValue("guestName", client.firstName);
      form.setValue("guestLastName", client.lastName || "");
      form.setValue("guestPhone", client.phone || "");
      form.setValue("guestEmail", client.email || "");
    }}
    onClear={() => {
      form.setValue("guestName", "");
      form.setValue("guestLastName", "");
      form.setValue("guestPhone", "");
      form.setValue("guestEmail", "");
    }}
  />
</div>
```

Crear el componente `SpaClientSelector` en el mismo archivo `spa.tsx` (o en un archivo separado):

```
function SpaClientSelector({ onSelect, onClear }: {
  onSelect: (client: SpaClient) => void;
  onClear: () => void;
}) {
  const [search, setSearch] = useState("");
  const [selected, setSelected] = useState<SpaClient | null>(null);
  const [open, setOpen] = useState(false);

  const { data: clients = [] } = useQuery<SpaClient[]>({
    queryKey: ["/api/spa/clients"],
  });

  const filtered = search.length >= 2
    ? clients.filter(c => {
        const s = search.toLowerCase();
        return c.firstName.toLowerCase().includes(s) ||
          (c.lastName || "").toLowerCase().includes(s) ||
          (c.phone || "").includes(s);
      })
    : [];

  if (selected) {
    return (
      <div className="flex items-center justify-between p-2 border rounded-md bg-
        <div>
          <p className="text-sm font-medium">{selected.firstName} {selected.lastN
            {selected.phone} && <p className="text-xs text-muted-foreground">{select
          </div>
          <Button variant="ghost" size="sm" className="h-6 text-xs" onClick={() =>
            Cambiar
          </Button>
        </div>
      );
    }

  return (
    <div className="relative">
      <Input
        placeholder="Buscar cliente por nombre o teléfono..."
        value={search}
        onChange={e => { setSearch(e.target.value); setOpen(true); }}
        onFocus={() => setOpen(true)}
      />
    </div>
  );
}
```

```

      data-testid="input-spa-client-search"
    />
    {open && filtered.length > 0 && (
      <div className="absolute z-50 w-full mt-1 bg-background border rounded-md
        {filtered.map(client => (
          <button
            key={client.id}
            className="w-full text-left px-3 py-2 text-sm hover:bg-muted transi
            onClick={() => { setSelected(client); setOpen(false); setSearch("")
            data-testid={`spa-client-option-${client.id}`}
          >
            <span className="font-medium">{client.firstName} {client.lastName |
              {client.phone && <span className="text-muted-foreground ml-2 text-x
            </button>
          )}}
        </div>
      )}
    </div>
  );
}

```

PARTE 3 — PDF de confirmación de turno SPA

Función `printSpaConfirmation`

Agregar en `spa.tsx`, cerca de donde está `setSelectedAppointment`:

```

const printSpaConfirmation = (appointment: SpaAppointment) => {
  const treatment = treatments.find(t => t.id === appointment.treatmentId);
  const cabin = cabins.find(c => c.id === appointment.cabinId);
  const professional = professionals.find(p => p.id === (appointment as any).prof
  const guestFullName = `${(appointment.guestLastName || "").toUpperCase()} ${app
  const appointmentDate = new Date(appointment.appointmentDate + "T12:00:00").toL
    weekday: "long", year: "numeric", month: "long", day: "numeric"
  });

  const html = `<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Confirmación Turno SPA - ${guestFullName}</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; font-size: 12px; color: #333; backgrou

```

```

.page { max-width: 780px; margin: 0 auto; padding: 0; }
.header-photo { width: 100%; height: 120px; background: linear-gradient(180deg, #4a7c72 0%, #f0f7f5 100%); }
.logo-bar { display: flex; align-items: center; justify-content: space-between; padding: 0 10px; }
.logo-left { display: flex; align-items: center; gap: 14px; }
.logo-circle { width: 56px; height: 56px; border-radius: 50%; border: 2px solid #4a7c72; }
.logo-name { font-size: 20px; font-weight: bold; color: #2d5a52; font-family: Georgia, serif; }
.logo-name span { display: block; font-size: 11px; font-weight: normal; color: #666; }
.content { padding: 24px 28px; }
.title-block { text-align: center; margin-bottom: 24px; padding: 16px; background-color: #f0f7f5; }
.title-block h2 { font-size: 18px; color: #2d5a52; font-family: Georgia, serif; margin: 0; }
.title-block p { font-size: 11px; color: #666; margin-top: 4px; }
.info-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 24px; }
.info-box { border: 1px solid #ddd; border-radius: 6px; padding: 12px; }
.info-box .info-label { font-size: 9px; color: #888; text-transform: uppercase; margin-bottom: 4px; }
.info-box .info-value { font-size: 13px; font-weight: bold; color: #111; margin-bottom: 4px; }
.info-box .info-sub { font-size: 10px; color: #666; margin-top: 2px; }
.detail-table { width: 100%; border-collapse: collapse; margin-bottom: 20px; }
.detail-table td { padding: 8px 12px; border: 1px solid #ddd; font-size: 11px; }
.detail-table .label-col { background-color: #f5f5f5; font-weight: bold; color: #555; padding: 8px 12px; }
.highlight-box { background-color: #f0f7f5; border: 2px solid #4a7c72; border-radius: 12px; padding: 16px; margin-bottom: 24px; }
.highlight-box .price { font-size: 24px; font-weight: bold; color: #2d5a52; margin-bottom: 4px; }
.highlight-box .price-label { font-size: 10px; color: #666; }
.conditions { font-size: 10px; color: #555; line-height: 1.7; margin-bottom: 24px; }
.conditions p::before { content: "» "; color: #4a7c72; }
.footer { border-top: 2px solid #4a7c72; padding: 12px 0 0 0; display: flex; align-items: center; justify-content: space-between; }
.footer-right { font-size: 13px; font-weight: bold; color: #2d5a52; letter-spacing: 0.5px; }
@media print { body { margin: 0; } .page { max-width: 100%; } }
</style>
</head>
<body>
<div class="page">
  <div class="header-photo">MARAN SPA & WELLNESS · PARANÁ</div>
  <div class="logo-bar">
    <div class="logo-left">
      <div class="logo-circle">Hotel<br>& Spa<br>MARAN<br>SUITES</div>
      <div class="logo-name">MARAN SPA & WELLNESS<span>Hotel & Spa · Paraíso</span></div>
    </div>
    <div style="font-size:10px; color:#888; text-align:right; line-height:1.8;">
      <strong style="color:#4a7c72;">CONFIRMACIÓN DE TURNO</strong><br>
      Reservado con éxito
    </div>
  </div>
  <div class="content">
    <div class="title-block">
      <h2>♦ Confirmación de Turno SPA ♦</h2>
      <p>Gracias por reservar. Te esperamos para una experiencia de bienestar única</p>
    </div>
  </div>
</div>

```

```

<div class="info-grid">
  <div class="info-box">
    <div class="info-label">Cliente</div>
    <div class="info-value">${guestFullName}</div>
    ${appointment.guestPhone ? `<div class="info-sub">📞 ${appointment.guestF
    ${appointment.guestEmail ? `<div class="info-sub">✉️ ${appointment.guestEm
  </div>
  <div class="info-box">
    <div class="info-label">Fecha y Hora</div>
    <div class="info-value">${appointment.startTime} hs</div>
    <div class="info-sub">${appointmentDate}</div>
  </div>
</div>
<table class="detail-table">
  <tr><td class="label-col">Tratamiento</td><td><strong>${treatment?.name ||
  <tr><td class="label-col">Duración</td><td>${treatment?.durationMinutes ||
  <tr><td class="label-col">Espacio</td><td>${cabin?.name || "-"}</td></tr>
  ${professional ? `<tr><td class="label-col">Profesional</td><td>${professio
  ${appointment.notes ? `<tr><td class="label-col">Notas</td><td>${appointmen
</table>
${treatment?.price ? `
<div class="highlight-box">
  <div class="price-label">Precio del servicio</div>
  <div class="price">$$${parseFloat(treatment.price).toLocaleString("es-AR", {
</div>` : ""}
<div class="conditions">
  <p>Por favor presentarse 10 minutos antes del turno.</p>
  <p>Ante cancelaciones, avisar con al menos 2 horas de anticipación.</p>
  <p>Recomendamos no consumir alimentos pesados previo al tratamiento.</p>
</div>
<div class="footer">
  <div style="line-height:1.8;">
    📍 Alameda de la Federación y Mitre, Paraná (3100), Entre Ríos<br>
    📞 +54 9 343 503 8070 &nbsp;|&nbsp;&nbsp;✉️ reservas@maran.com.ar<br>
    🌐 maran.com.ar
  </div>
  <div class="footer-right">MARAN SPA</div>
</div>
</div>
</body>
</html>`;

```

```

const win = window.open("", "_blank", "width=820,height=700");
if (win) {
  win.document.write(html);
  win.document.close();
}

```

```

        win.focus();
        setTimeout(() => win.print(), 500);
    }
};

```

Botón en el detalle del turno

En el panel de detalle del turno (`selectedAppointment`), agregar el botón de confirmación junto a los botones de editar/cancelar:

```

<Button
  variant="outline"
  size="sm"
  onClick={() => printSpaConfirmation(selectedAppointment)}
  data-testid="button-print-spa-confirmation"
>
  <Printer className="h-4 w-4 mr-1" />
  Confirmación
</Button>

```

Agregar `Printer` a los imports de `lucide-react` si no está.

Resumen de archivos modificados

Archivo	Cambio
<code>shared/schema.ts</code>	Nueva tabla <code>spaClients</code>
<code>server/migrate.ts</code>	CREATE TABLE <code>spa_clients</code>
<code>server/routes.ts</code>	4 endpoints nuevos para clientes SPA
<code>client/src/pages/spa.tsx</code>	Autocomplete habitación, selector cliente SPA, función PDF turno, botón imprimir
<code>client/src/pages/spa-clients.tsx</code>	Nueva página (crear)
<code>client/src/App.tsx</code>	Ruta <code>/spa-clients</code>
<code>AppSidebar</code>	Ítem "Clientes SPA" debajo de SPA

